SPM Module 2

1 Introduction

1.1 Partition Phase

I initially analyzed the partitioning phase, which processes N elements (where N is the cardinality of either relation R or S). This phase can be broken down into three steps: histogram computation, prefix sum, and scatter.

Histogram Phase For each record, this step performs one `compute_partition_id` evaluation followed by a memory update (`hist[pid]++`), for a total of $O(N)$ operations. Updates are applied to an array of size P of 64-bit elements. For typical values of P, this array fits in cache, leading to low-latency accesses despite the random `pid` generation. A key factor is the impact of increasing P: as the array grows, it may exceed cache capacity, turning fast accesses into cache misses.

From a parallelization perspective, histogram computation is a good candidate for parallel execution, since the input relation can be evenly partitioned across threads. However, concurrent updates require care: using a shared histogram would need synchronization (mutexes, atomic operations, etc.), which can introduce thread contention and false sharing. Alternatively, one can use thread-local histograms that are merged into a global one at the end of the phase; this will be discussed in detail later.

Prefix Sum Phase The `exclusive_prefix_sum` phase iterates over an array of size P (the number of partitions). Since typically $P \ll N$, its computational cost is negligible in practice.

Parallelizing the prefix sum would introduce thread creation and synchronization overhead that would outweigh any potential speedup. For this reason, this step is not parallelized.

Scatter Phase Like the histogram, the scatter phase processes N elements and performs $O(N)$ operations. The key difference is the memory access pattern. It evaluates the partition ID and performs a store plus increment (`out[next[pid]++]`) into an output array of size N, where reasonably N is much larger than P. Consecutive tuples are mapped to widely dispersed, non-contiguous memory locations, failing to exploit spatial locality, making this phase more expensive.

The access pattern depends on the relation between data volume (N) and number of partitions (P). In particular, the ratio N/P defines the average partition size (assuming uniformity) and thus the average distance between the P active write cursors. For large N, these cursors span distant memory regions separated by roughly N/P elements. As the loop alternates writes across these regions, spatial locality is lost.

Given its high cost, the scatter phase is a natural candidate for parallelization. While the workload can be distributed in a lock-free manner, the underlying limitation remains hardware-bound: due to irregular memory accesses over a large footprint, performance is constrained by memory bandwidth rather than compute capacity.

1.2 Local Join Phase

The local join phase is executed independently for each of the P partitions. For a given partition p, the algorithm performs a hash join between R_p and S_p using a build-and-probe strategy.

This phase is the most naturally parallelizable. Once the relations are partitioned, each partition join is fully independent, enabling embarrassingly parallel execution where partitions are assigned to threads without requiring locks or synchronization.

Build Phase This step scans R_p to construct a lookup structure for fast access (again $O(N)$ operations). The reference implementation uses a `std::unordered_map` to store key multiplicities and handle duplicates.

Standard `unordered_map` implementations are hardware-inefficient due to their node-based, separate-chaining design. Even with `countR.reserve(...)` to pre-allocate buckets (i.e., the array of pointers), node storage is still allocated dynamically per entry. In case of collisions, updates require pointer chasing over non-contiguous memory, degrading spatial locality. The goal is to retain $O(1)$ average lookup complexity while improving cache locality and reducing pointer-based indirection.

Probe Phase After building the hash map for R_p, the algorithm scans S_p once (again $O(N)$ operations). For each record, it performs a lookup in the `unordered_map` to check membership in R_p. If a match is found, join counters and checksums are updated according to key multiplicity. As in the build phase, performance is limited by pointer chasing in the hash table, since `find` operations may traverse non-contiguous chains in memory.

1.3 Data Structure Optimization

As established previously, the `std::unordered_map` baseline severely limits throughput in the local join phase. Its main limitation is the *separate chaining* (node-based) design: it introduces overhead from repeated dynamic allocations for inserted keys and forces unpredictable pointer chasing during lookups.

To bridge the gap between algorithmic complexity and hardware performance, it is necessary to move from closed addressing to an *open addressing* scheme. Open addressing eliminates these bottlenecks by storing keys and values in a single contiguous bucket array, avoiding external node allocations.

Among modern implementations, I selected `ska::flat_hash_map` for its high-performance, header-only design, enabling plug-and-play integration without external dependencies.

Technically, a single `reserve()` call removes per-element allocation overhead for the entire partition. Its Robin Hood hashing with linear probing resolves collisions by scanning contiguous slots, keeping related entries spatially close and reducing cache misses.

This design involves an explicit space–time trade-off. To maintain a low load factor and ensure $O(1)$ probe complexity, `ska::flat_hash_map` pre-allocates a structure larger than strictly necessary: with `reserve(n)`, the internal array typically grows to about $2n$ entries. As a result, construction and destruction are more expensive than for `std::unordered_map`, which grows incrementally via node allocation. In this setting, where a fresh map is created and destroyed for each of the P partitions, this overhead becomes measurable and is explicitly evaluated below.

1.3.1 Empirical Evaluation of Data Structure Optimization [EXPERIMENT 0]

To validate the benefits of open addressing, the baseline (`std::unordered_map`) and optimized (`ska::flat_hash_map`) sequential implementations were benchmarked under identical conditions. The test uses $N = 20 \times 10^6$ records per relation, $P = 256$ partitions, and uniformly distributed keys in $[0, 5 \times 10^6)$, yielding an average multiplicity of $N/\text{max\_key} = 4$. This setup stresses both collision resolution and multi-match handling.

With an average partition size of $N/P \approx 78{,}000$, each `reserve()` allocation produces a hash table of about 1.78 MB per partition (156,000 slots $\times$ 12 bytes). This exceeds L1d (32 KiB) and L2 (256 KiB) caches of the Intel Xeon E5-2640 v2, but remains within the 20 MiB L3 cache.

Execution times are reported in Table 1, as median values over 10 runs (first 2 discarded as warm-up).

Regarding instrumentation: the *Build* timer starts after map construction and `reserve()`, and stops before destruction. The *Join Loop* timer instead includes per-partition overhead not captured individually in Build and Probe, namely repeated `reserve()` and destructor calls across all $P = 256$ partitions. This explains why the Join Loop speedup (1.62×) is lower than Build (3.41×) and Probe (2.79×). $\Delta_{\texttt{std}} = 1389.62 - (684.67 + 544.27) = 160.68$ ms

$$\Delta_{\texttt{ska}} = 856.93 - (200.94 + 194.82) = 461.17 \text{ ms}$$

As expected, the gains originate from improved memory locality, while the additional cost stems from larger upfront allocations. In the current design this cost is incurred P times; in the parallel version, it will be amortized by allocating the structure once and avoiding repeated construction and destruction.

Table 1: Performance Comparison: `std::unordered_map` vs `ska::flat_hash_map`

Phase	**Baseline (`std`)** Median (ms) (Std)	**Optimized (`ska`)** Median (ms) (Std)	**Speedup**
Partitioning R			
Histogram	36.18 (0.13)	36.26 (0.14)	-
Scatter	333.27 (2.98)	329.11 (0.81)	-
Total R	369.45 (2.90)	365.55 (0.82)	-
Partitioning S			
Histogram	36.23 (0.12)	36.12 (0.20)	-
Scatter	333.71 (2.82)	327.00 (0.75)	-
Total S	369.96 (2.77)	363.42 (0.79)	-
Join Phase (timers exclude `reserve()` and destructor overhead)			
Build	684.67 (0.91)	200.94 (0.21)	**3.41**×
Probe	544.27 (0.92)	194.82 (0.23)	**2.79**×
Join Loop†	**1389.62 (1.51)**	**856.93 (1.52)**	**1.62**×
Global Total	**2130.22 (4.50)**	**1586.80 (1.99)**	**1.34**×

† Join Loop includes `reserve()` and destructor cost for all P partitions, not captured by the Build/Probe timers. The unmeasured overhead is ≈161 ms for `std` and ≈461 ms for `ska`.

The results confirm the theoretical predictions. The 3.41× speedup on Build directly reflects the elimination of per-element dynamic allocation and pointer indirection. The 2.79× speedup on Probe confirms that open-addressing lookup, which scans mostly adjacent memory slots, generates significantly fewer cache misses than `std`'s pointer-chasing traversal of linked chains.

1.4 Local Join Parallelization Strategy [EXPERIMENT 1]

1.4.1 Workload Characterization

As previously discussed, the local join phase operates on P independent partitions, making partition-level distribution the main factor for parallel efficiency. During the build phase, each R_p is scanned to construct a hash table with nominal $O(|R_p|)$ complexity under constant-time insertions. In practice, performance depends on key cardinality: fewer unique keys reduce state size, increase duplication, and lower both memory footprint and insertion overhead.

The probe phase sequentially scans S_p, performing one hash lookup per element. If a match is found, the stored multiplicity is used to update join counters and checksums. Thus, probe complexity is $O(|S_p|)$, with one lookup per tuple.

Overall cost per partition depends jointly on partition size and key diversity in R_p: build cost is driven by key uniqueness, while probe cost scales with $|S_p|$.

Under uniform key generation and an approximately uniform `compute_partition_id`, partitions receive similar loads and collision rates (depending on `max_key`), yielding near-balanced work distribution—crucial for scalability.

To evaluate scheduling behavior, three strategies were implemented: Static Block-Based, Static Block-Cyclic, and Dynamic Thread-Pool.

All parallel executions avoid shared mutable state in inner loops. Each thread uses thread-local accumulators (`local_total`, `local_timing`) on its own stack, writing results only after completing assigned partitions. To prevent false sharing, results are stored in vectors of `AlignedJoinResult` and `AlignedPhaseTiming`, padded to 64-byte cache-line alignment.

Chunk Size Tuning and Analysis [EXPERIMENT 1.1] A hyperparameter sweep over `chunk_size` was performed for Block-Cyclic and Dynamic scheduling. Results are reported in Table 2.

Table 2: Join Loop Median Time (ms, std over 10 runs) — $N = 20$M, $P = 256$, N_THREADS = 16

Strategy	1	2	4	8	16	32	64
Block-Cyclic	231.47 (7.80)	233.64 (7.81)	234.68 (12.68)	226.66 (10.60)	**207.05 (6.45)**	242.49 (8.41)	355.21 (39.73)
Dynamic Pool	230.71 (24.56)	221.29 (6.83)	244.72 (18.08)	237.88 (10.24)	263.49 (15.39)	262.23 (20.15)	390.94 (32.17)

1.4.2 Result [EXPERIMENT 1.2]

Table 3: Join Loop Execution Time (ms, std over 10 runs) — $N = 20$M, $P = 256$, N_THREADS = 16

Strategy	Median (ms)	Std
Block-Cyclic(16)	207.05	6.45
Dynamic Pool(2)	221.29	6.83
Block-Based	228.998	10.320

Although a minor workload imbalance naturally occurs, both the input data and the compute_partition_id function distribute elements across partitions fairly uniformly. This inherent balance allows the Static Block-Cyclic strategy to perform best in this controlled setup. However, a critical look at the results reveals that the standard deviations overlap, meaning the performance gap between the static and dynamic approaches is not absolutely definitive. I decided to select the Block-Cyclic strategy for the current nearly uniform scenario; nevertheless, in more realistic contexts where key distributions are often skewed, a dynamic load-balancing solution like the Thread-Pool would likely be superior.

1.4.3 Memory Allocation Hoisting Optimization [EXPERIMENT 1.3]

As shown previously, the large gap between the total `join_loop` time (≈ 211 ms) and the sum of `build` and `probe` phases (≈ 104 ms) was caused by repeated dynamic memory allocations across all P partitions.

To remove this overhead, I introduced an *allocation hoisting* optimization. Instead of creating and destroying a `ska::flat_hash_map` per partition, each thread now allocates a single map locally, outside the processing loop. The map is initialized once using `reserve((R.size() / p) * 2)`, where the $2\times$ factor provides headroom for skew and prevents rehashing.

This thread-local structure is passed by reference to the join routine and cleared between iterations. Importantly, `clear()` resets the logical size while preserving the allocated capacity, avoiding further allocations.

The impact is substantial: the median `join_loop` time decreases from **211.69 ms** to **150.08 ms**. By amortizing allocation costs to a single per-thread initialization, the parallel phase achieves an additional **29% speedup**, bringing runtime closer to the intrinsic cost of the build and probe operations.

1.5 Partitioning Parallelization Strategy [EXPERIMENT 2]

1.5.1 Compute Histogram [EXPERIMENT 2.1]

In this phase, workload balancing is not a concern. The input consists of a flat array of records, and the histogram computation applies the same constant-time operation to each element: compute the partition identifier via a hash function and increment the corresponding counter. So a simple block partition between threads ensures an even distribution of work.

Naive Atomic Approach A simple way to parallelize this procedure is to split the input into equal blocks and assign one to each thread. Every thread processes its records and updates a single shared histogram using atomic increments (`std::atomic::fetch_add`). While this ensures a correct result without using complex locks, it scales poorly because multiple threads constantly fight to update the same memory locations. This competition, combined with false sharing—where different counters sit on the same cache line—creates a massive bottleneck on the memory bus that significantly slows down the entire process. Experimental results over 10 runs ($N = 2^{24}, p = 256, n\_threads = 16$) confirm this inefficiency: the atomic histogram phase takes 173.73 ± 1.96 ms, compared to the sequential baseline of 36.18 ± 0.13 ms. This results in a negative speedup of approximately $0.21\times$, meaning the parallel execution is nearly five times slower than the single-threaded version due to extreme synchronization overhead.

Thread-Local Privatization To resolve the performance issues of the naive method, we privatize the histogram using a nested structure (`std::vector<std::vectorstd::size_t>`), where each thread is assigned its own independent counter array. By allocating these buffers separately on the heap, we eliminate the need for expensive atomic operations and significantly reduce false sharing, as the memory regions for each thread are no longer strictly contiguous. Each thread populates its local histogram using standard non-atomic increments without any synchronization overhead. Once the parallel phase is complete, a final sequential reduction merges the local results into the global output. Benchmark results over 10 runs ($N = 2^{24}, p = 256, n\_threads = 16$) show a dramatic improvement: the histogram phase for relation R takes only 6.13 ± 0.21 ms. This achieves a speedup of approximately $5.9\times$ over the sequential baseline and is nearly $28\times$ faster than the naive atomic approach.

Result Recap

Table 4: Performance Summary for Histogram Computation (10 runs)

Approach	P	Median	(Std)	Speedup
Sequential Baseline	256	36.18 ms	(0.13)	1.00×
Naive Atomic	256	173.02 ms	(1.96)	0.21×
Privatized	256	6.13 ms	(0.21)	5.90×

(a) General results ($N = 2^{24}$, p = 256, 16 threads)

1.5.2 Scatter Phase [EXPERIMENT 2.2]

In this phase, as we know, the objective is to physically reorganize the flat input array so that records belonging to the same partition become contiguous in memory. It is characterized by pseudo-random memory writes, as the destination of each record is dictated by its hash value, making cache utilization and memory bandwidth the primary performance bottlenecks.

Naive Atomic Approach A straightforward parallelization strategy divides the input array into equal chunks, relying on atomic increments (`fetch_add`) on shared cursors to determine the global output position for each partition. While functionally simple, this approach exhibits abysmal scalability due to two severe hardware-level bottlenecks. First, concurrent atomic updates to the same partition cursor trigger intense cache line ping-ponging, as CPU cores constantly invalidate each other's L1 caches to gain exclusive ownership (atomic contention). Second, because the $p = 256$ cursors are stored contiguously, cursors for different partitions inevitably fall into the same 64-byte cache line; thus, even independent writes to distinct partitions force cache invalidations across cores. Consequently, threads stall almost entirely on memory synchronization rather than actual data

movement. Empirical results ($N = 2 \cdot 10^7, p = 256, n\_threads = 16$ over 10 runs) highlight this crippling overhead, with the scatter phase dominating execution time at 235.43 ± 1.39 ms.

Thread-Local Offsets To eradicate the severe overhead of atomic operations, we replace the global 1D prefix sum with a 2D prefix sum derived from thread-local histograms. Instead of first reducing all local histograms into a single global histogram, we preserve the per-thread partition counts, since an early reduction would lose the information required to assign disjoint write regions to each thread. Previously, threads competed for a single shared cursor per partition, necessitating hardware locks. The new approach iterates through partitions and then threads, pre-calculating exact, mutually exclusive memory bounds for each thread within every partition (`thread_offsets[t][pid]`). By distributing these private write cursors, threads can scatter their data entirely independently, reducing memory synchronization to zero. Benchmark results reflect this architectural shift, with scatter time plummeting to 66.65 ± 0.52 ms, a massive speedup over the atomic baseline. However, a new microarchitectural bottleneck emerges: because records are still written individually to seemingly random partition offsets, the CPU executes scattered, scalar writes. This highly non-sequential memory access pattern inherently defeats the L1 hardware prefetcher, resulting in elevated cache miss rates and sub-optimal memory bus utilization.

Result Recap

Table 5: Performance Comparison: Scatter Phase (Sequential vs Parallel) [$N = 2^{24}$, p = 256, 16 threads, 10 runs]

Phase	**Baseline (ska)**	**Parallel Atomic**		**Parallel Standard**	
	Median (Std) [ms]	Median (Std) [ms]	Speedup	Median (Std) [ms]	Speedup
Scatter R	329.11 (0.81)	230.44 (5.80)	**1.43**×	66.65 (0.52)	**4.94**×
Scatter S	327.00 (0.75)	235.43 (2.97)	**1.39**×	65.66 (0.53)	**4.98**×

Note: Speedup is calculated relative to the baseline (ska). The "Parallel Standard" version shows a significant reduction in memory usage compared to the "Atomic" version.

2 Scalability [EXPERIMENT 3]

2.1 Strong Scalability Analysis [EXPERIMENT 3.1]

Test setup: $N_R = N_S = 20 \times 10^6$, Max Key $= 5 \times 10^6$, $P = 256$, Chunk $= 16$, Node07. The median time was reported, out of 10 runs.

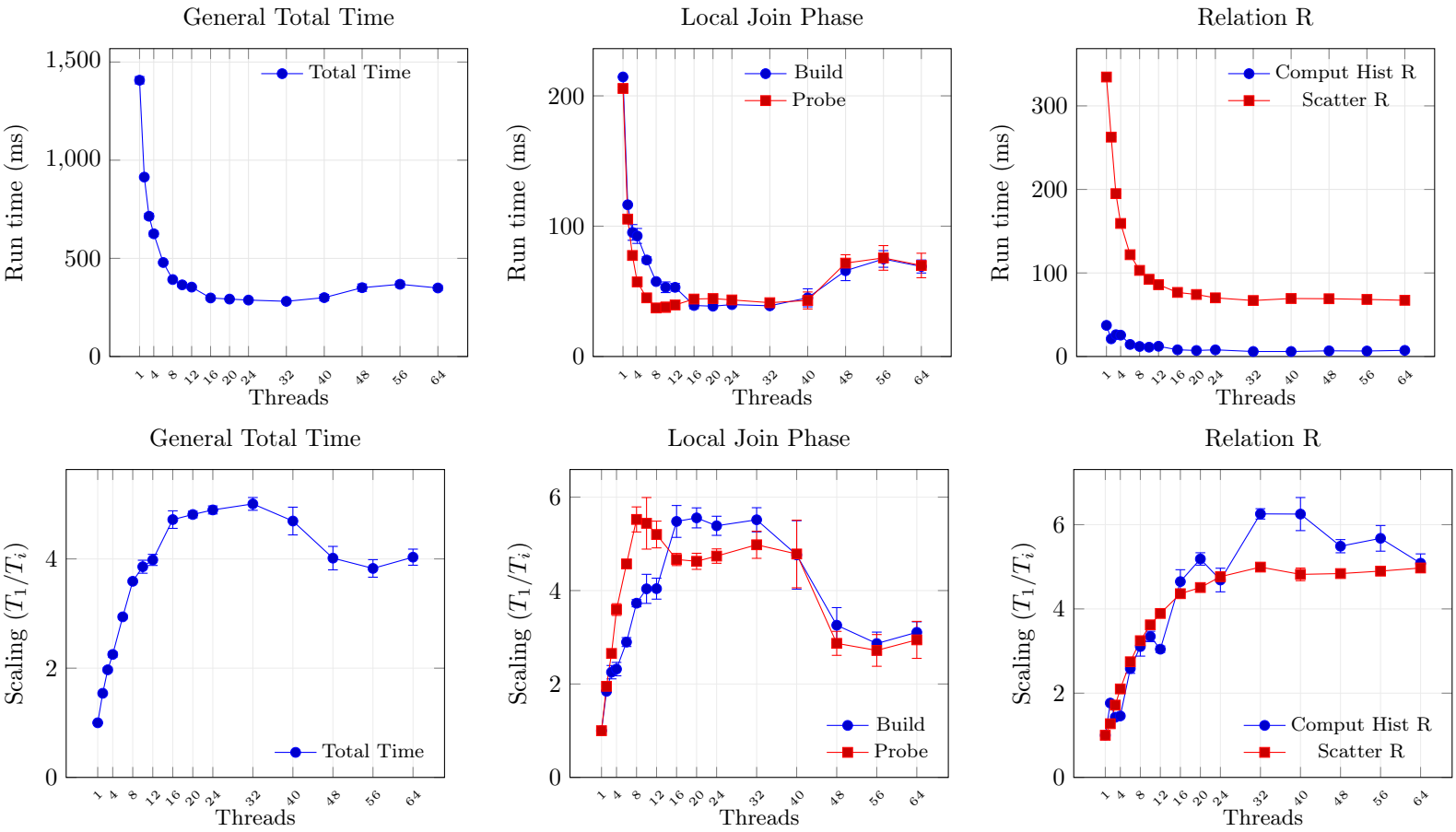

Figure 1: Speedup vs. Number of Threads. The Y-axis shows the ratio T_1/T_i. Error bars denote propagated uncertainty. Note: Speedup variance is calculated as $\Delta S = T_1 \cdot \frac{\Delta T_i}{T_i^2}$, treating the single-thread baseline T_1 as a constant reference.

2.2 Weak Scalability Analysis

To evaluate weak scalability, the goal is to maintain a strictly constant workload per thread across the algorithm's three main kernels. For the histogram and scatter phases, the operation count depends only on N/T, enabling linear scaling independently of the number of partitions P or the `max_key` range. The local join phase instead requires a more careful parameterization: each thread processes P/T partitions of average size N/P, resulting in a constant baseline workload of N/T elements.

During the build phase, each thread scans its assigned tuples and performs hash map accesses, balancing cheaper counter increments with more expensive insertions. Under a uniform distribution, this ratio depends directly on the average key multiplicity; therefore, the $N/$`max_key` ratio must remain constant to preserve the per-thread build cost. Similarly, the probe

the probe phase executes exactly N/T lookups, while the frequency of successful matches, and the associated checksum updates, is stabilized by maintaining the same uniform distribution and $N/$`max_key` ratio.

Beyond the logical operation count, the partition size (N/P) is also kept constant to preserve as much as possible cache hierarchy behavior. Thus, the benchmark parameters are scaled as follows:

$$\begin{cases} N_T = N_1 \cdot T & \text{(Total problem size)} \\ P_T = P_1 \cdot T & \text{(Total partitions)} \\ K_T = K_1 \cdot T & \text{(Key range, where } K = \text{max\_key)} \end{cases} \tag{1}$$

Base test setup ($T = 1$): $N_1 = 2 \times 10^6$, Max Key $K_1 = 5 \times 10^5$, $P_1 = 64$, Chunk = 16, Node07. The median time was reported, out of 10 runs.

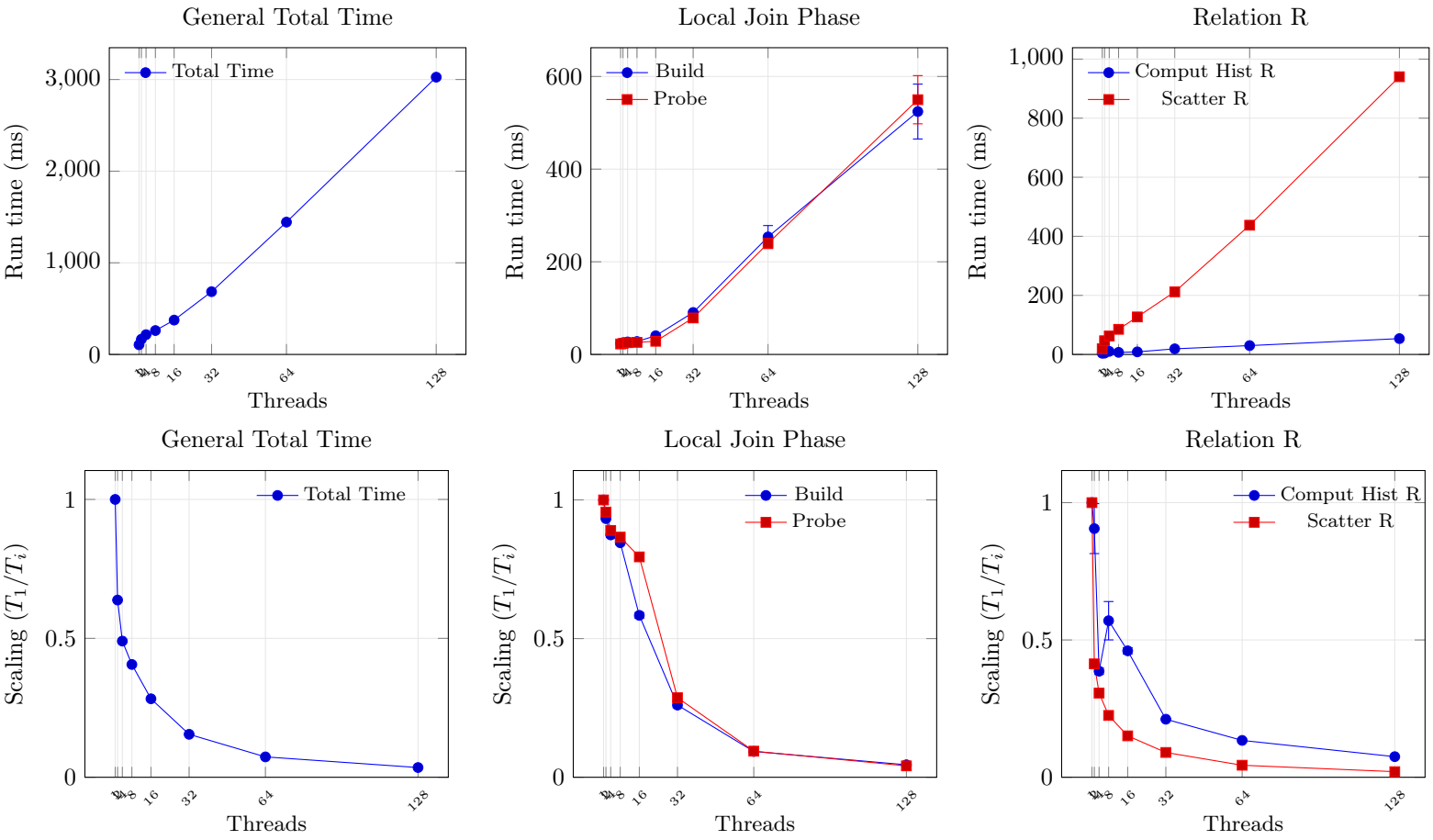

Figure 1: Speedup vs. Number of Threads. The Y-axis shows the ratio T_1/T_i. Error bars denote propagated uncertainty. Note: Speedup variance is calculated as $\Delta S = T_1 \cdot \frac{\Delta T_i}{T_i^2}$, treating the single-thread baseline T_1 as a constant reference.

Comment The empirical results closely match the theoretical expectations for a memory-intensive partitioned hash join under weak scaling. By scaling N, P, and `max_key` proportionally with the thread count T, the computational density and cache pressure per thread remain approximately constant, as confirmed by the nearly stable execution times of the *Build* and *Probe* kernels up to $T = 16$. In contrast, the *Scatter* phase shows a predictable non-linear latency increase due to memory-bandwidth saturation and higher TLB pressure as the number of active write cursors grows with P. The sharp degradation for $T \geq 32$ indicates the transition to a hardware-limited regime, where physical cores are exhausted and Hyper-Threading contention introduces additional overhead.

Correctness Verification Strategy To guarantee correctness without degrading parallel performance, the implementation uses a lock-free local accumulation approach where each thread computes its partial `join_count` and checksums using strictly private variables. This is mathematically safe because the operations updating the metrics, specifically the integer additions for the match count and the checksum accumulations, are strictly associative and commutative. Consequently, the main thread can safely sum the independent partial results regardless of the threads' execution order. To physically prevent parallel threads from invalidating each other's CPU cache lines (false sharing) when storing these results, the output array uses 64-byte padded structures (`alignas(64)`). Finally, the verification is kept strictly separate from performance profiling: small datasets ($N_R, N_S \leq 500$) undergo an exhaustive $\mathcal{O}(|R| \times |S|)$ cross-check, while larger datasets validate execution integrity by confirming that the aggregated signatures perfectly match the sequential baseline.

1 Conclusion

Table 1: Performance Comparison: Sequential Baseline (`std`) vs Sequential Optimized (`ska`) vs Parallel Final

Phase	**Baseline (`std`)**	**Optimized (`ska`)**	**Parallel Final**	**Speedup (Parallel vs)**	
	Median (ms) (Std)	Median (ms) (Std)	Median (ms) (Std)	**Base**	**Opt**
Partitioning R					
Histogram	36.18 (0.13)	36.26 (0.14)	5.98 (0.30)	6.05×	6.06×
Scatter	333.27 (2.98)	329.11 (0.81)	67.03 (0.34)	4.97×	4.91×
Total R	369.45 (2.90)	365.55 (0.82)	73.27 (0.36)	**5.04**×	**4.99**×
Partitioning S					
Histogram	36.23 (0.12)	36.12 (0.20)	6.69 (0.35)	5.41×	5.40×
Scatter	333.71 (2.82)	327.00 (0.75)	66.00 (0.26)	5.06×	4.95×
Total S	369.96 (2.77)	363.42 (0.79)	72.66 (0.37)	**5.09**×	**5.00**×
Join Phase (timers exclude `reserve()` and destructor overhead)					
Build	684.67 (0.91)	200.94 (0.21)	47.28 (2.12)	**14.48**×	**4.25**×
Probe	544.27 (0.92)	194.82 (0.23)	55.88 (2.21)	**9.74**×	**3.49**×
Join Loop[†]	**1389.62 (1.51)**	**856.93 (1.52)**	**150.43 (9.98)**	**9.24**×	**5.70**×
Global Total	**2130.22 (4.50)**	**1586.80 (1.99)**	**297.46 (9.83)**	**7.16**×	**5.33**×

[†] Join Loop includes `reserve()` and destructor cost for all P partitions, not captured by the Build/Probe timers. The unmeasured sequential overhead is ≈161 ms for `std` and ≈461 ms for `ska`.